

 DAY 19:
JAVASCRIPT TIMING
EVENTS
(SETTIMEOUT,
SETINTERVAL,
CLEARTIMEOUT,
CLEARINTERVAL)

1. setTimeout() – Execute a Function After a Delay

The setTimeout() function executes a function once after a specified time (in milliseconds).



Syntax:

```
setTimeout(function, delay);
```

- ◆ delay is in milliseconds (1 second = 1000ms).

✓ Example: Delayed Message (2 seconds)

```
console.log("Start");  
setTimeout(() => {  
    console.log("Hello after 2 seconds!");  
}, 2000);  
console.log("End");
```

📌 Output (after 2 sec delay):

```
Start  
End  
Hello after 2 seconds!
```

🔧 Use Case: Showing popups, lazy loading, delaying API calls.

2. setInterval() – Execute a Function Repeatedly

The setInterval() method executes a function repeatedly at a fixed interval until stopped.



Syntax:

```
setInterval(function, interval);
```

- ◆ The function runs indefinitely until manually stopped using clearInterval().



Example: Show Time Every Second

```
setInterval(() => {  
    console.log(`Current Time: ${new  
Date().toLocaleTimeString()}`);  
}, 1000);
```

 Output (every second):

```
Current Time: 10:00:01 AM
```

```
Current Time: 10:00:02 AM
```

```
Current Time: 10:00:03 AM
```

```
...
```

 Use Case: Digital clocks, real-time updates, refreshing data.

3. clearTimeout() – Stop a Scheduled setTimeout()

If you set a timeout but later decide not to execute it, use clearTimeout().

✓ Example: Cancel Timeout Before Execution

```
let timeoutId = setTimeout(() => {  
    console.log("This will not run!");  
}, 5000);  
  
clearTimeout(timeoutId);
```

 Output:

Nothing happens because clearTimeout(timeoutId) cancels it.

4. clearInterval() – Stop a Repeating setInterval()

If you want to stop an interval-based function, use clearInterval().

✓ Example: Stop a Counter After 5 Seconds

```
let counter = 0;
let intervalId = setInterval(() => {
  counter++;
  console.log(`Counter: ${counter}`);
  if (counter === 5) {
    clearInterval(intervalId);
  }
}, 1000);
```

// Stops after 5 executions
console.log("Counter Stopped!");

Output:

```
Counter: 1  
Counter: 2  
Counter: 3  
Counter: 4  
Counter: 5  
Counter Stopped!
```

 Use Case: Stopping animations, live updates, polling APIs.